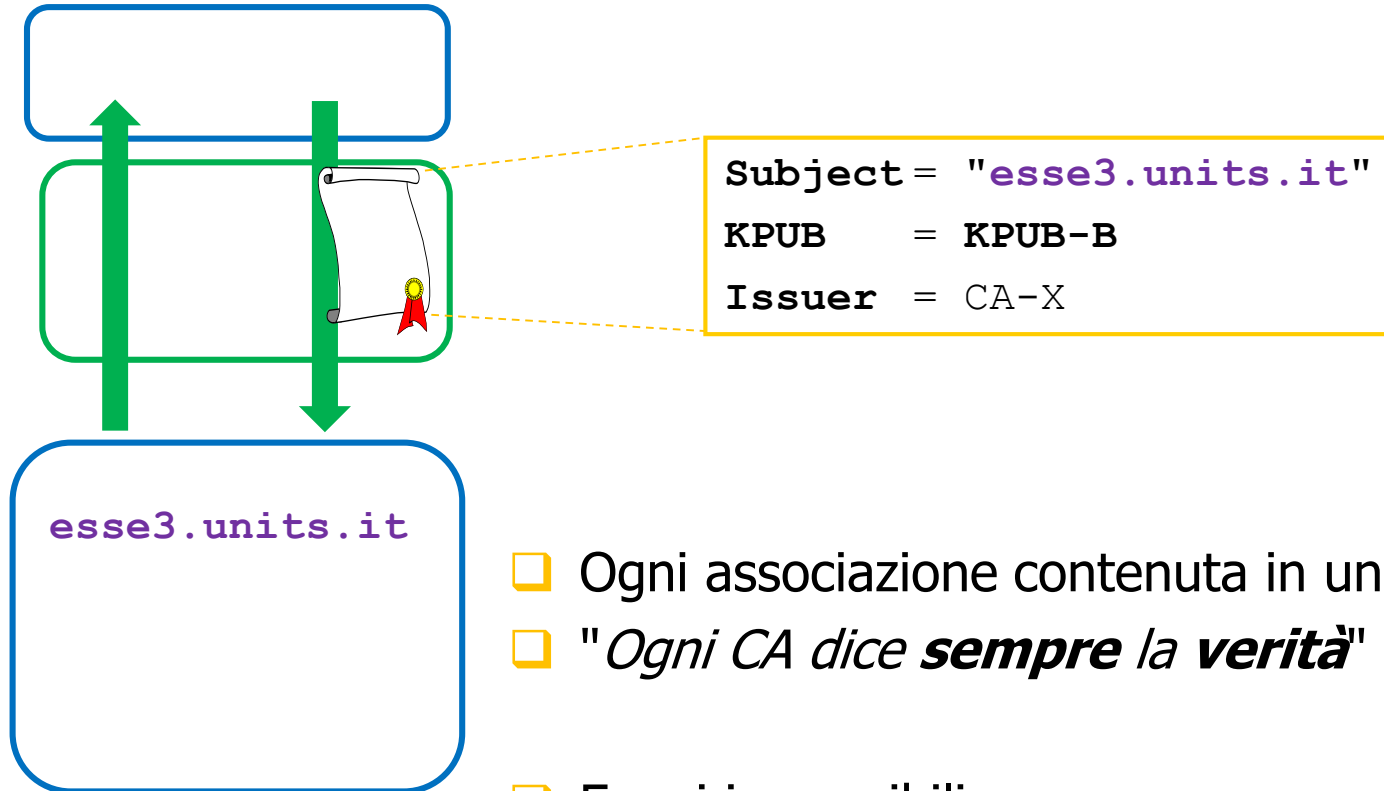


Trusted CA



REMIND

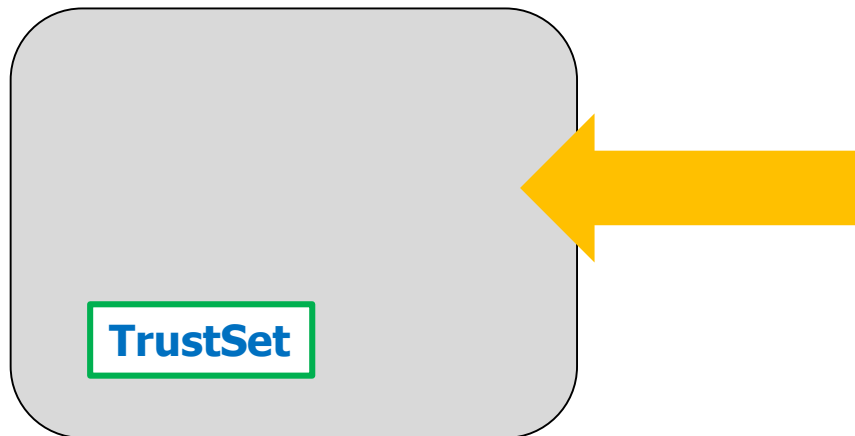
Atto di fede



- ❑ Ogni associazione contenuta in un certificato è **vera**
- ❑ "Ogni CA dice ***sempre la verità***"
- ❑ Errori impossibili
- ❑ Frodi impossibili

REMININD **Atto di fede:** SOLO per ALCUNE CA

- ❑ Ogni associazione contenuta in un certificato è **vera**
- ❑ *"Ogni CA dice sempre la verità"*
- ❑ **Solo** per le CA il cui nome è in un **TrustSet** predefinito
 - ❑ Altro punto di vista: **mi fido** solo delle CA nel mio TrustSet



Subject= esse3.units.it
KPUB = KPUB-E
Issuer= CA-X

Subject= Bartoli Alberto
KPUB = KPUB-B
Issuer= CA-Y

Verifica Certificato

- ❑ $CERT = \langle S = \text{"Ancelotti"}, KPUB = KPUB - M, I = CA - X \rangle$
è autentico e integro
AND
- ❑ $CA - X$ certifica solo associazioni vere



- ❑ $KPUB - M$ è la chiave pubblica di



E se Realtà $\neq$ Ipotesi ?

□ CERT=<S="Ancelotti", KPUB=KPUB-M, I=CA-X>
è autentico e integro

AND

□ ~~CA-X~~ certifica solo associazioni vere

CA-X certifica associazioni vere ed associazioni false



□ KPUB-M è la chiave pubblica di



Ovvio!

- ❑ $CERT = \langle S = \text{"Ancelotti"}, KPUB = KPUB - M, I = CA - X \rangle$
è autentico e integro

AND

- ~~❑ $CA - X$ certifica solo associazioni vere~~
 $CA - X$ certifica associazioni vere ed associazioni false



- ❑ $KPUB - M$ può essere la chiave pubblica di
o può non esserlo
- ❑ **Nessuna garanzia**



Importantissimo



- ❑ Inserire CA-X in TrustSet fornisce a CA-X un potere **enorme**
 - ❑ I sw che usano quel TrustSet **assumono vere tutte le associazioni** <Subject-KPUB> emesse da CA-X

- ❑ Inserire CA-X in TrustSet di **moltissimi dispositivi** fornisce a CA-X un potere ancora più grande!

Nota bene (I)

Kazakhstan government is now intercepting all HTTPS traffic

Written by Catalin Cimpanu,



Starting Wednesday, July 17, 2019, the Kazakhstan government has started intercepting all HTTPS internet traffic inside its borders.

Local internet service providers (ISPs) have been instructed by the local government to force their respective users into **installing a government-issued certificate on all devices, and in every browser.**

Nota bene (II)



Russia creates its own TLS certificate authority to
bypass sanctions

BLEEPINGCOMPUTER

March 10, 2022

Russia has created its own trusted TLS certificate authority (CA) to solve website access problems that have been piling up after sanctions prevent certificate renewals.

The sanctions imposed by western companies and governments are preventing Russian sites from renewing existing TLS certificates, causing browsers to block access to sites with expired certificates.

Nota bene (III)



Last Chance to fix eIDAS: Secret EU law threatens Internet security

moz://a

Over 400 cyber security experts, researchers and NGOs sign an [open letter](#) sounding the alarm

2nd November 2023

All web browsers distributed in Europe will be required to trust the certificate authorities and cryptographic keys selected by EU governments

This enables the government **of any EU member state** to issue website certificates for interception and surveillance which can be used against **every EU citizen**

...even those **not** resident in or connected to the issuing member state

Hhmmm....

Come faccio a **essere certo** che CA-X
certifica **solo** associazioni vere?

- ☐ NO Certificazioni fraudolente
- ☐ NO Errori procedurali
- ☐ NO Attacchi informatici
- ☐ ...



KPRIV-A
KPUB-A



Subject= "Ancelotti"
KPUB = KPUB-M
Issuer= CA-X



KPRIV-M
KPUB-M

NON SI PUO'!!!



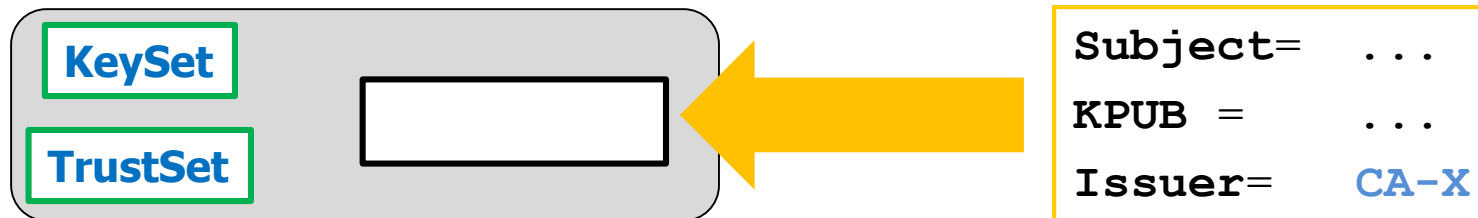
- ❑ "*CA-X* certifica solo associazioni vere" è una affermazione che **assumo vera** (**atto di fede**)
- ❑ Esattamente come l'**ipotesi** di un teorema matematico

- ❑ IF ipotesi
vera **nella realtà**
- ❑ THEN deduzioni basate sulla ipotesi
vere **nella realtà**
- ❑ ELSE deduzione basate sulla ipotesi
possono essere vere o possono essere false **nella realtà**

Verifica certificati (SENZA bacchetta magica)



Verifica certificato (REMININD)



❑ Ogni sw che **riceve** un certificato lo **verifica**

1. $CA-X \in \text{TrustSet}$

2. $CA-X \in \text{KeySet}$

3. Autentico ed integro (bacchetta magica)

❑ Se tutti i test sono superati (certificato **valido**)

❑ Allora **usa** il certificato, procedura prosegue

❑ Altrimenti **non** usa il certificato, procedura abortisce

Come si toglie la magia?



KeySet:

Non nomi, bensì KPUB



- Ogni sw che usa crittografia a chiave pubblica usa:
 - ...
 - TrustSet
 - Nomi CA di cui si fida
 - KeySet
 - ~~Nomi~~ **KPUB** CA di cui può verificare auth+integr dei certificati
- Strutture dati **predefinite**
- Per il momento **non sappiamo come inizializzate**

Auth + Integrity grazie a Firma Digitale

□ **Emissione** Certificato: **Firmato dello Issuer**

□ **Validazione** Certificato:

1. $CA-X \in \text{TrustSet}$
2. $CA-X \in \text{KeySet}$
3. Autentico ed integro
(~~bacchetta magica~~) (verifica firma su CERT)

□ Necessaria KPUB di Issuer...ce l'ho in **KeySet!**

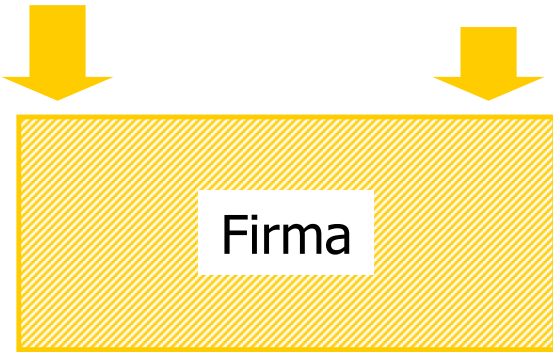


Emissione Certificato

Ogni certificato è **firmato** dallo Issuer

Subject= esse3.units.it
KPUB = KPUB-E
Issuer= CA-X

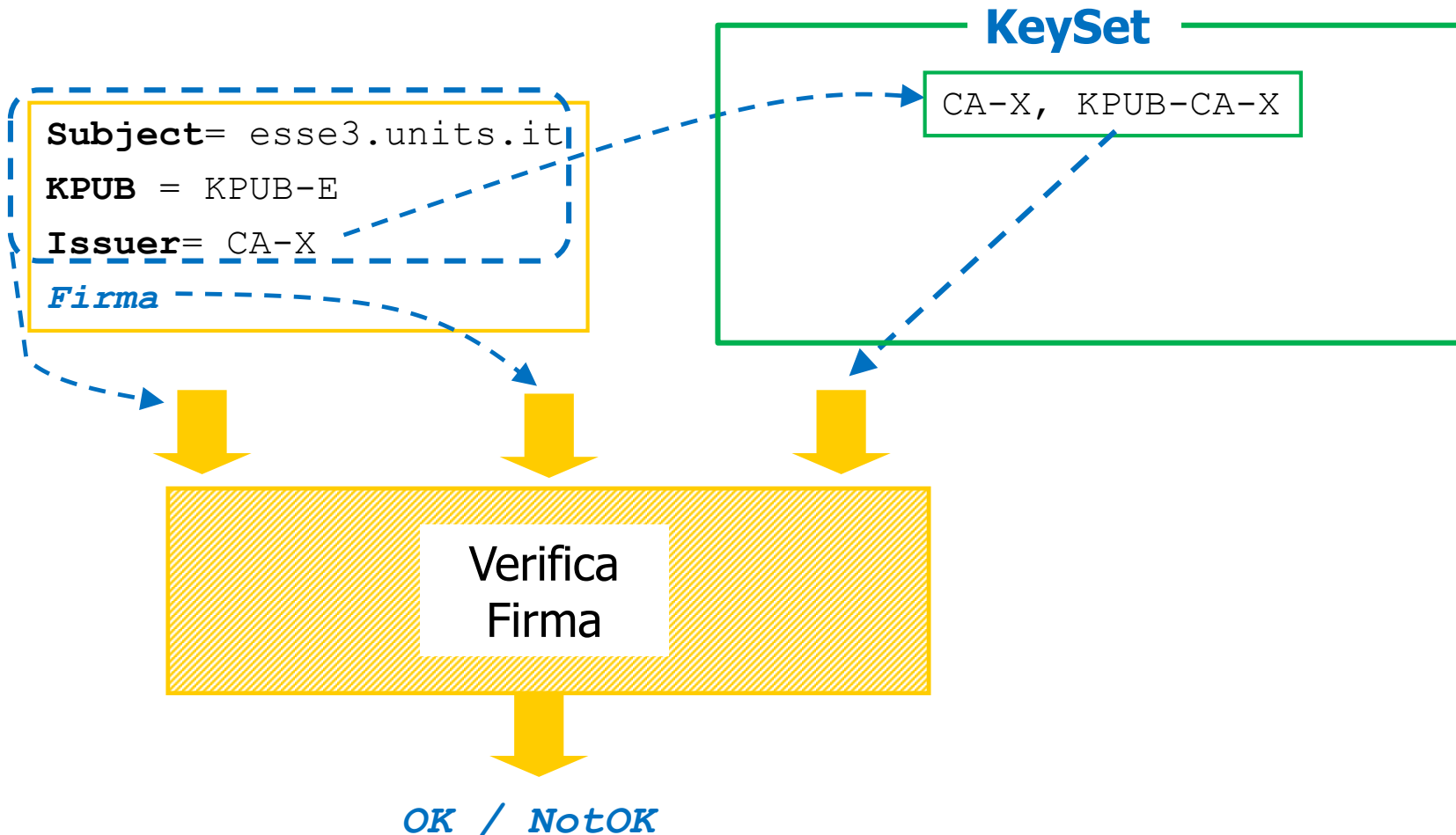
KPRIV-CA-X



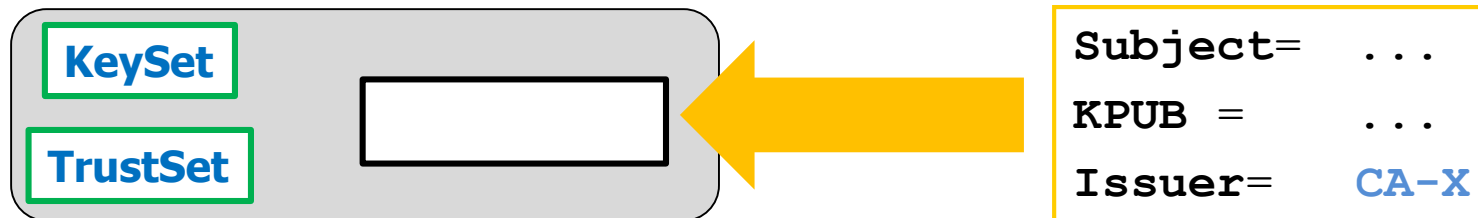
Subject= esse3.units.it
KPUB = KPUB-E
Issuer= CA-X
Firma

CA-X

Verifica Firma Certificato



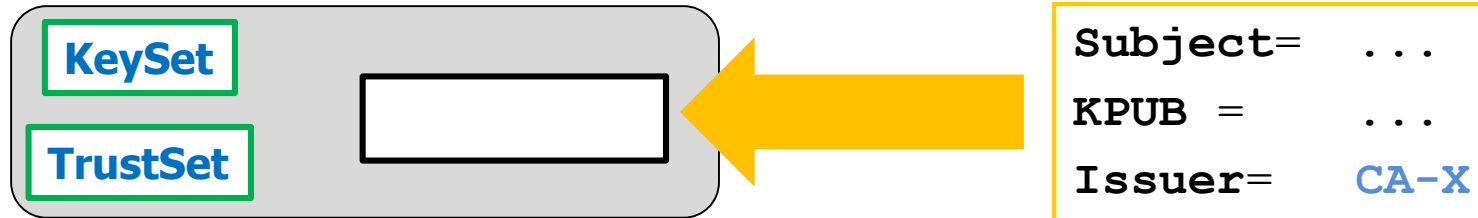
Validazione certificato (SENZA BACCHETTA MAGICA)



□ Ogni sw che **riceve** un certificato lo **verifica**

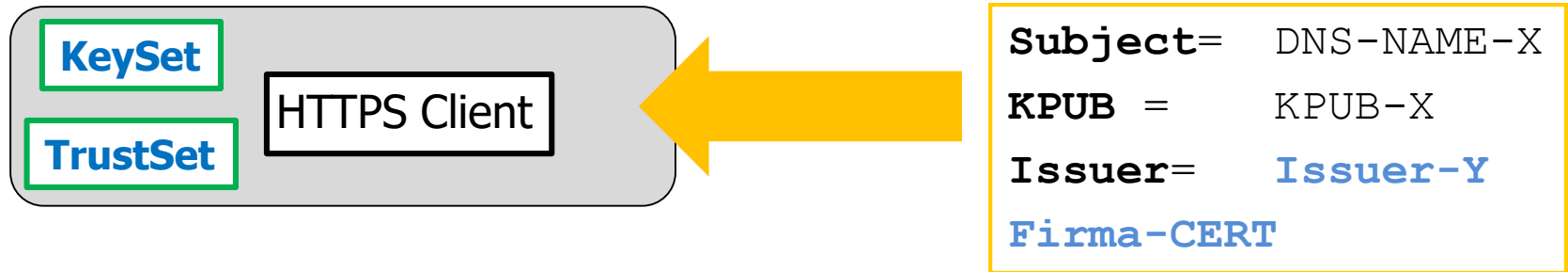
1. $CA-X \in \text{TrustSet}$
2. $CA-X \in \text{KeySet}$
3. Autentico ed integro
(~~bacchetta magica~~)
(verifica firma su CERT con $KPUB = \text{KeySet}(CA-X)$)

Esempi



- ❑ Ogni sw che **riceve** un certificato lo **verifica**
- ❑ Seguono due **esempi di uso dei certificati**
(in pratica ci sono molti altri usi)
 - ❑ Autenticazione Server in HTTPS
 - ❑ Firma digitale su file

Esempio: Cert. per HTTPS Authentication (I)

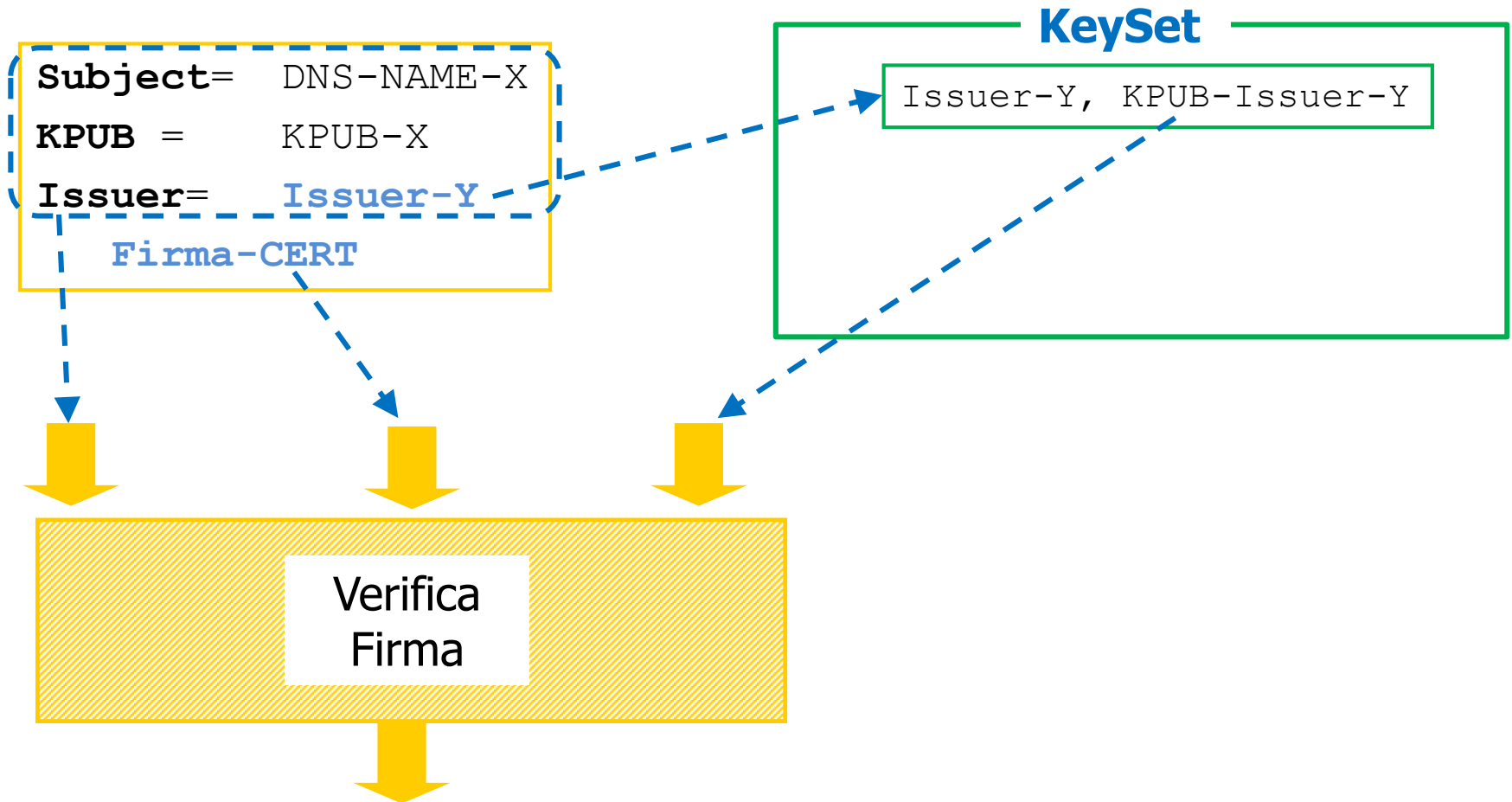


□ HTTPS Client :

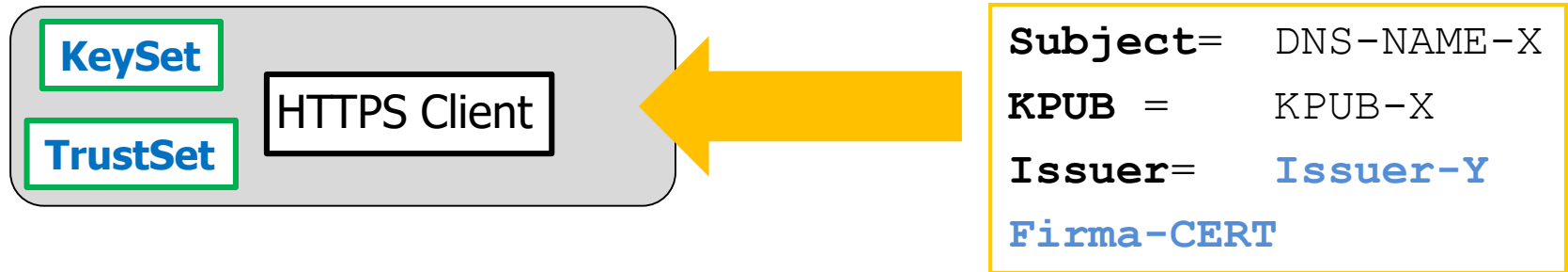
- Deve validare CERT(...) per essere certo che KPUB-X è veramente la chiave pubblica di DNS-NAME-X

1. CA-X ∈ TrustSet
2. CA-X ∈ KeySet
3. Autentico ed integro (prossima slide)

Esempio: Cert. per HTTPS Authentication (II)



Esempio: Cert per HTTPS Authentication (III)

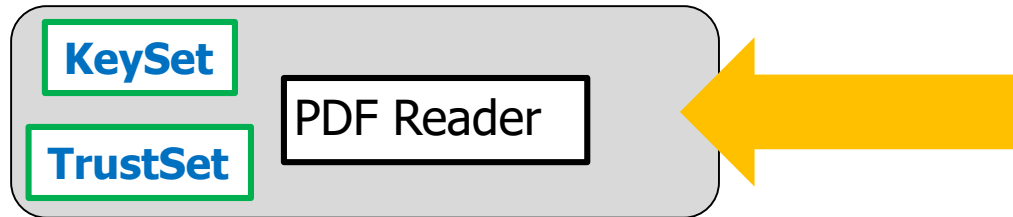


□ HTTPS Client :

- Deve validare CERT(...) per essere certo che KPUB-X è veramente la chiave pubblica di DNS-NAME-X
- CERT(...) valido
⇒ **può proseguire con le azioni TLS** assumendo che KPUB-X è veramente la chiave pubblica di DNS-NAME-X

Esempio:

Verifica Firma Digitale (I)



PDF-Byte-Sequence
Firma-PDF-ByteSeq
"Firmato da Subject-X"



Subject= Subject-X
KPUB = KPUB-X
Issuer= **Issuer-Y**
Firma-CERT

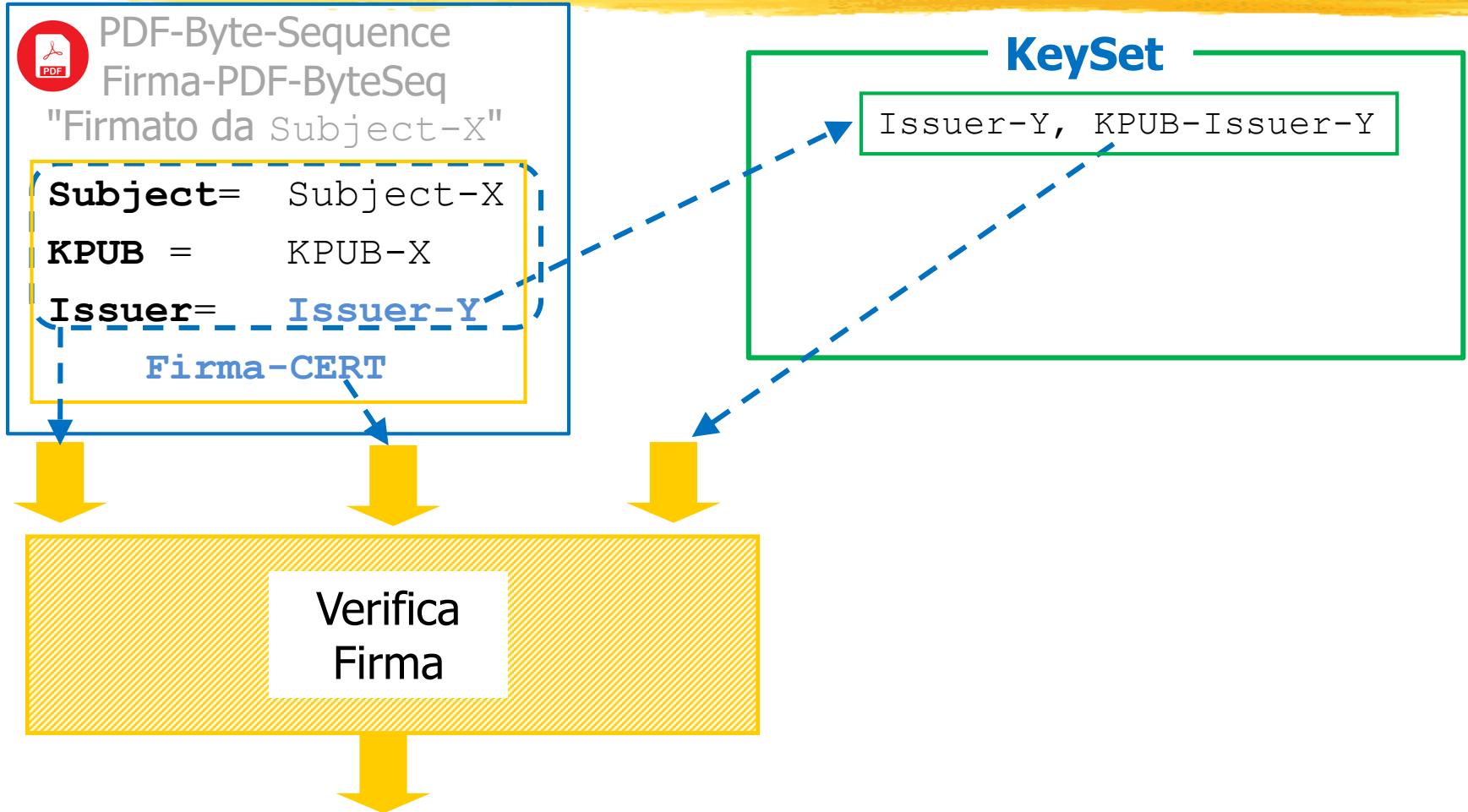
□ PDF Reader :

- Deve validare CERT(...) per essere certo che KPUB-X è veramente la chiave pubblica di Subject-X

1. CA-X $\in$ TrustSet
2. CA-X $\in$ KeySet
3. Autentico ed integro (prossima slide)

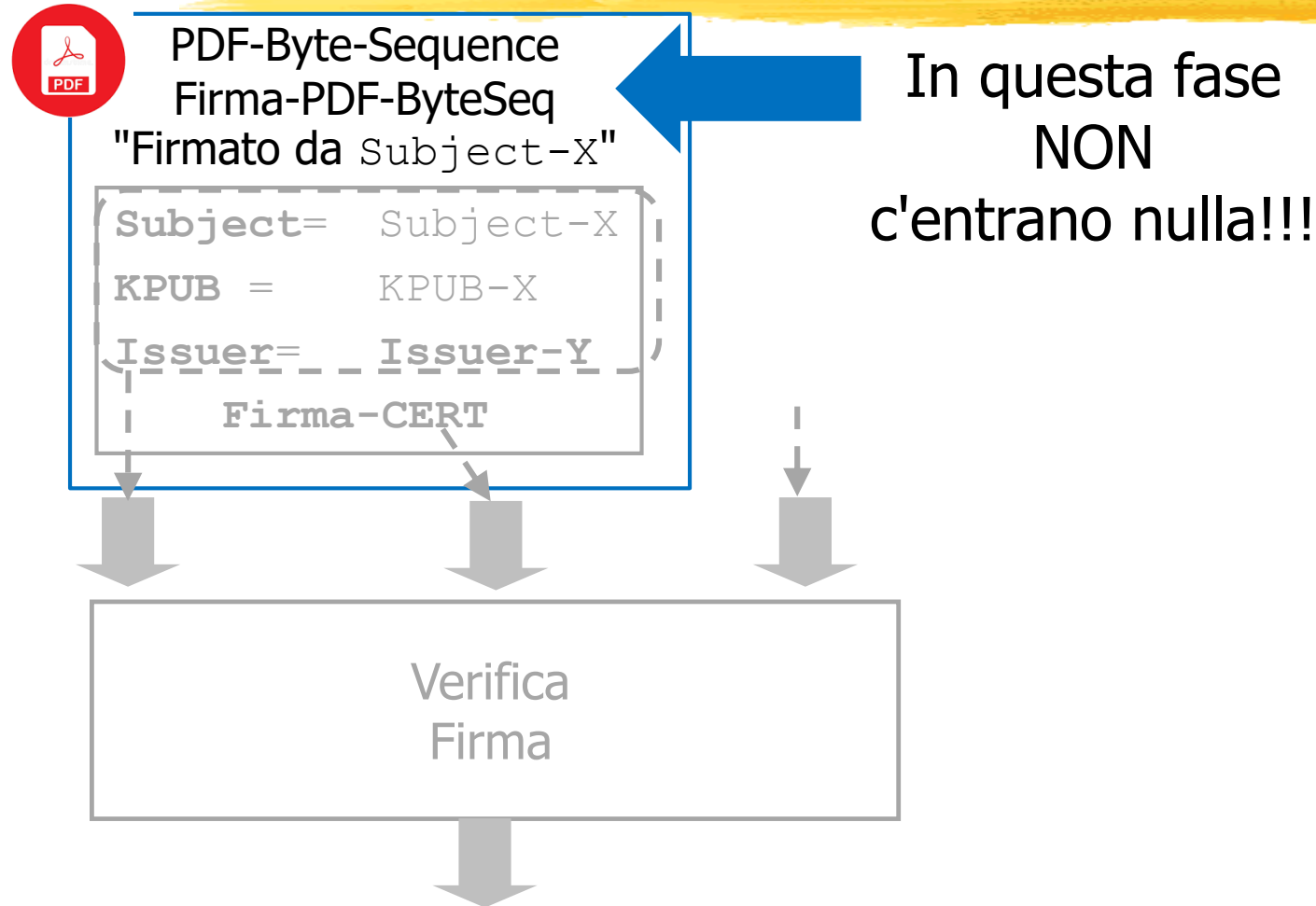
Esempio:

Verifica Firma Digitale (II-a)



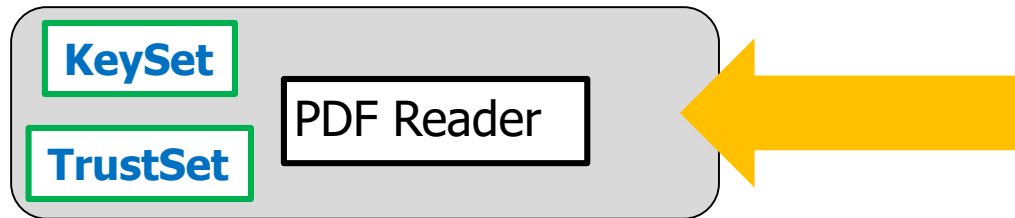
Esempio:

Verifica Firma Digitale (II-b)



Esempio:

Verifica Firma Digitale (III)



PDF-Byte-Sequence
Firma-PDF-ByteSeq
"Firmato da Subject-X"



Subject= Subject-X
KPUB = KPUB-X
Issuer= Issuer-Y
Firma-CERT

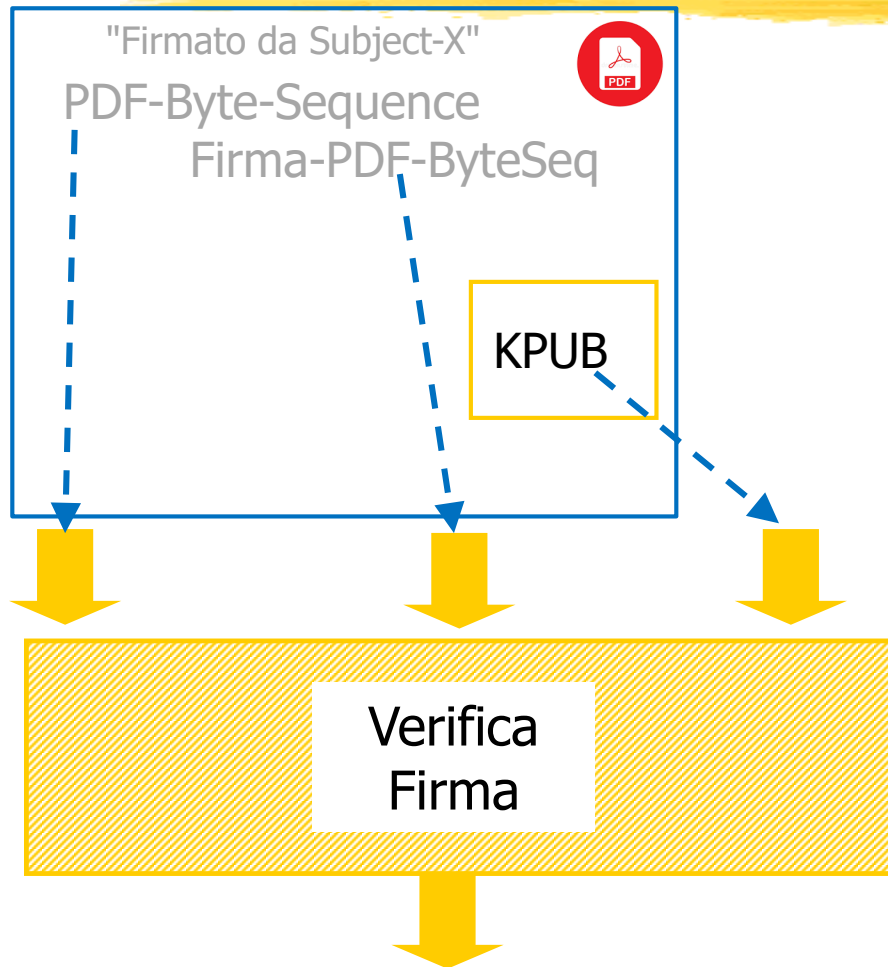
□ PDF Reader :

□ Deve validare CERT(...) per essere certo che
KPUB-X è veramente la chiave pubblica di Subject-X

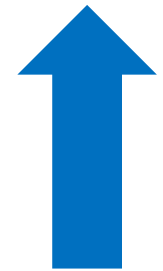
□ CERT(...) valido

⇒ **può proseguire con le azioni di verifica firma** assumendo che
KPUB-X è veramente la chiave pubblica di DNS-NAME-X

Come prosegue...

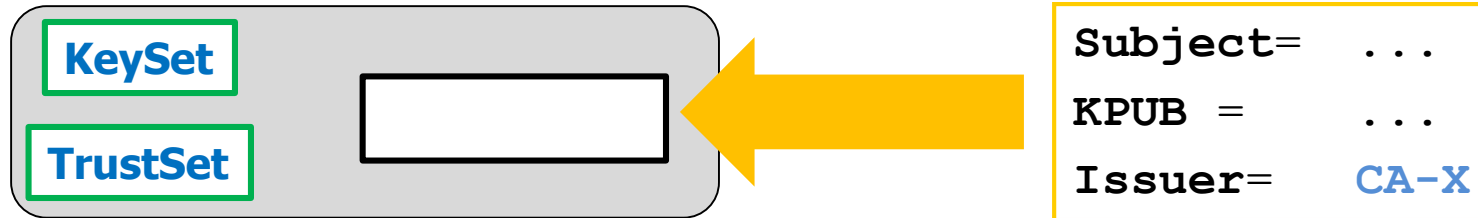


KeySet



Adesso NON
serve più!!!

Ripeto



❑ Ogni sw che **riceve** un certificato lo **verifica**

1. $CA-X \in \text{TrustSet}$
2. $CA-X \in \text{KeySet}$
3. Autentico ed integro
(verifica firma su CERT con $KPUB = \text{KeySet}(CA-X)$)

❑ Le azioni sono **sempre le stesse** (vedi esempi precedenti)

KeySet / TrustSet in pratica



KeySet, TrustSet: Inizializzazione

KeySet

CA-X, KPUB-X

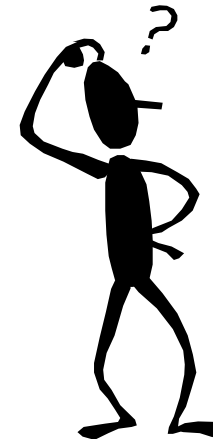
CA-Y, KPUB-Y

TrustSet

CA-X

CA-Y

- *E' importante che la **configurazione** (inizializzazione) sia "corretta"?*



KeySet, TrustSet: Protezione

KeySet

CA-X, KPUB-X

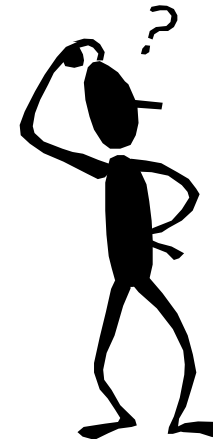
CA-Y, KPUB-Y

TrustSet

CA-X

CA-Y

- ☐ Devo proteggerle in **scrittura**?
- ☐ Devo proteggerle in **lettura**?



KeySet, TrustSet: Gestione



- ❑ Inizializzazione:
 - ❑ Vediamo dopo
- ❑ Protezione accessi **software**:
 - ❑ Sistema operativo
(SubjectFisico si autentica come Subject)
- ❑ Protezione accessi **fisici**:
 - ❑ If a bad guy has physical access to your computer,
then it is not your computer anymore

KeySet, TrustSet: Cosa contengono

Cosa potrebbero contenere **in teoria**

KeySet

CA-X, KPUB-X

CA-Y, KPUB-Y

TrustSet

CA-X

CA-Y

Cosa contengono **in realtà**:
Certificati **self-signed**
(sw più semplice)

KeySet

Subject= CA-X

KPUB = KPUB-X

Issuer= CA-X

Subject= CA-Y

KPUB = KPUB-Y

Issuer= CA-Y

TrustSet

Subject= CA-X

KPUB = KPUB-X

Issuer= CA-X

Subject= CA-Y

KPUB = KPUB-Y

Issuer= CA-Y

Dove sono? (REMINDE)



1. PersonalSet

2. TrustSet

3. KeySet

☐ Esistono **sempre**

☐ **Non** hanno questo nome

☐ Spesso sono **mescolati** tra loro

☐ Locazione, Nome, Formato dipendono dal sw/s.o.
(vedremo)

Dove sono



- ❑ Si trovano in **files**
- ❑ Diversi in funzione del **software** che li usa
 - ❑ Windows:
Control Panel → Opzioni Internet → Connessioni → Certificati
 - ❑ Chrome: usa questi ma "will soon use its own" (October 2020)
 - ❑ Firefox ha i propri separati da quelli di Windows
 - ❑ Software Java:
Files con un certo nome in una certa directory sotto JAVA_HOME
 - ❑ GnuPG
Files con un certo nome in una certa directory sotto GNUPG_HOME
 - ❑ Android:
Impostazioni → Sicurezza → Credenziali attendibili

Contenuto di default:

Esempio



Java

- ❑ KeySet 5 elementi (Verisign e pochi altri)
- ❑ TrustSet empty

Windows

- ❑ KeySet circa 30 elementi
- ❑ TrustSet sottoinsieme del KeySet ("Trusted Root CA": quasi tutto)

OpenSSL

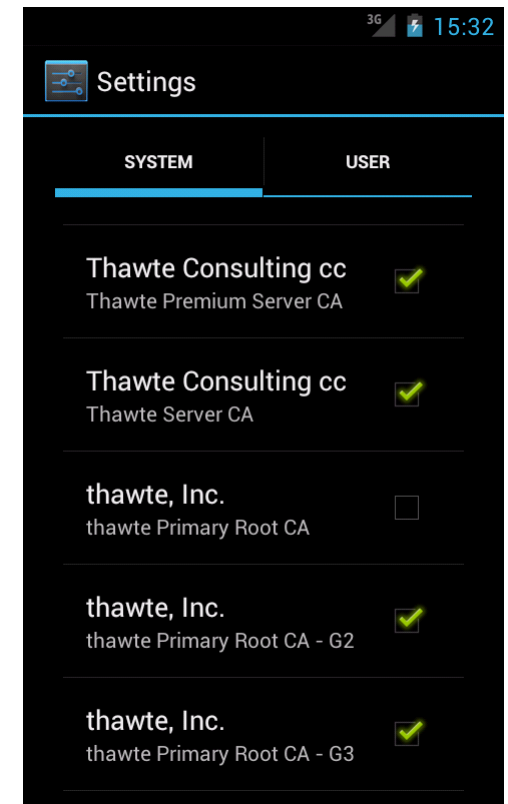
- ❑ Uso di KeySet / TrustSet molto complesso (noi non li abbiamo usati)

Android

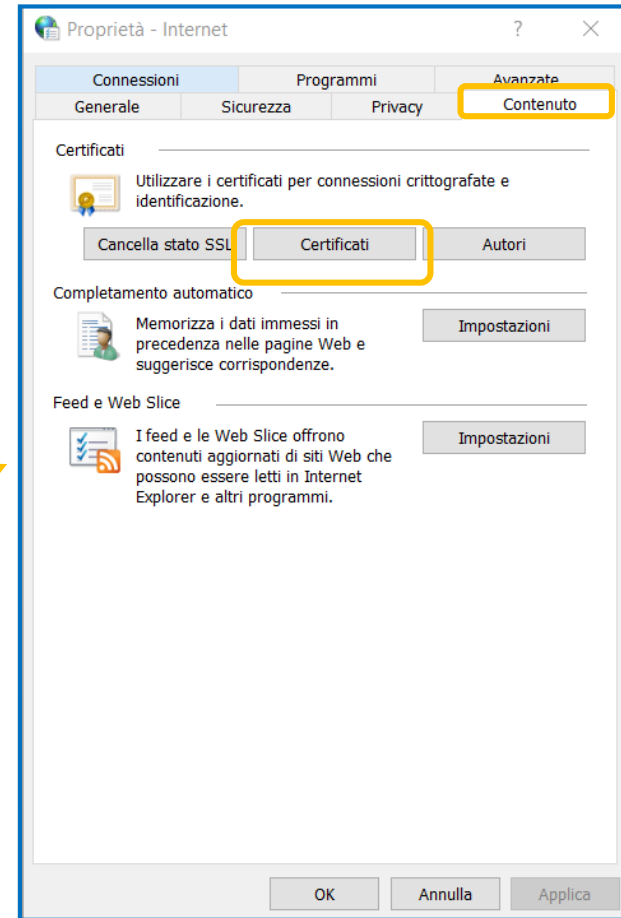
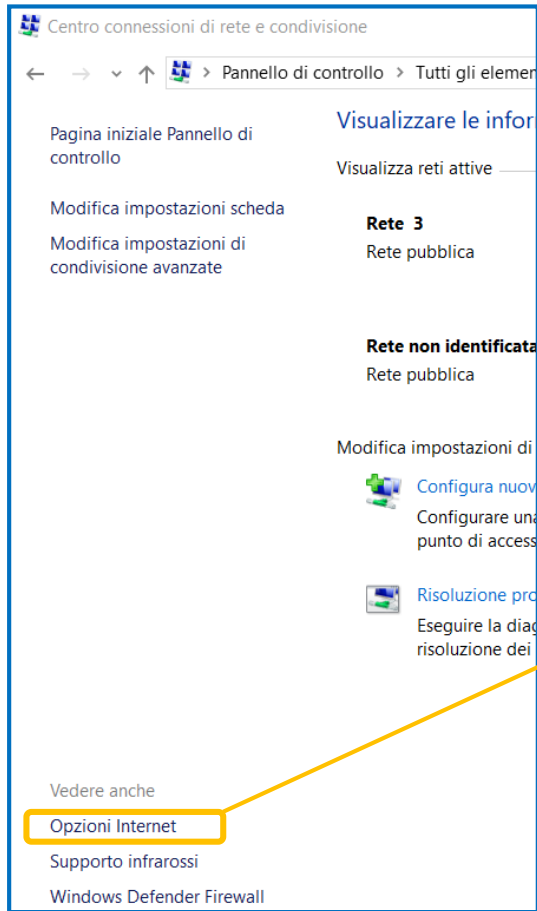
❑ Impostazioni → Sicurezza → Credenziali attendibili

❑ Provate a contarli

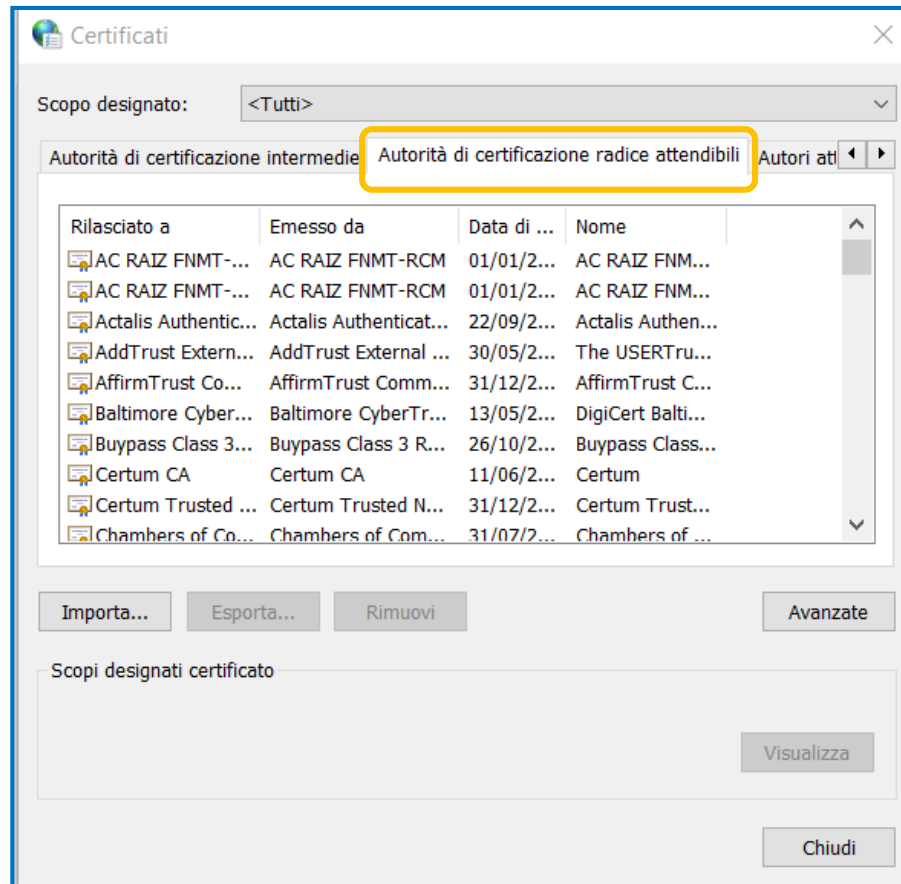
❑ Poi chiedetevi "di chi mi sto fidando?"



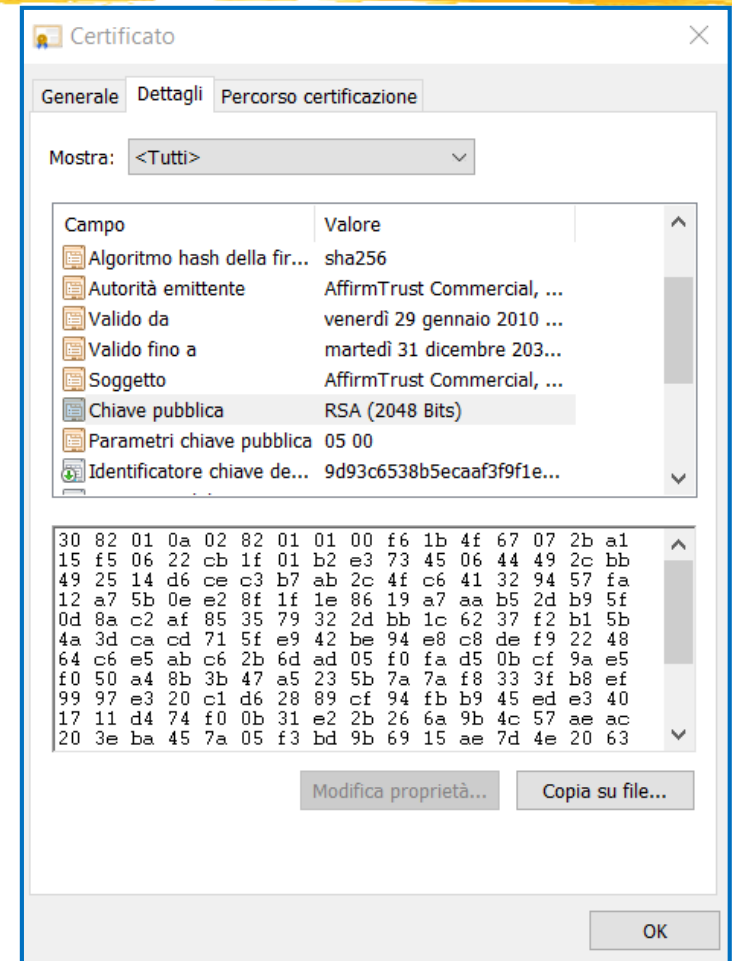
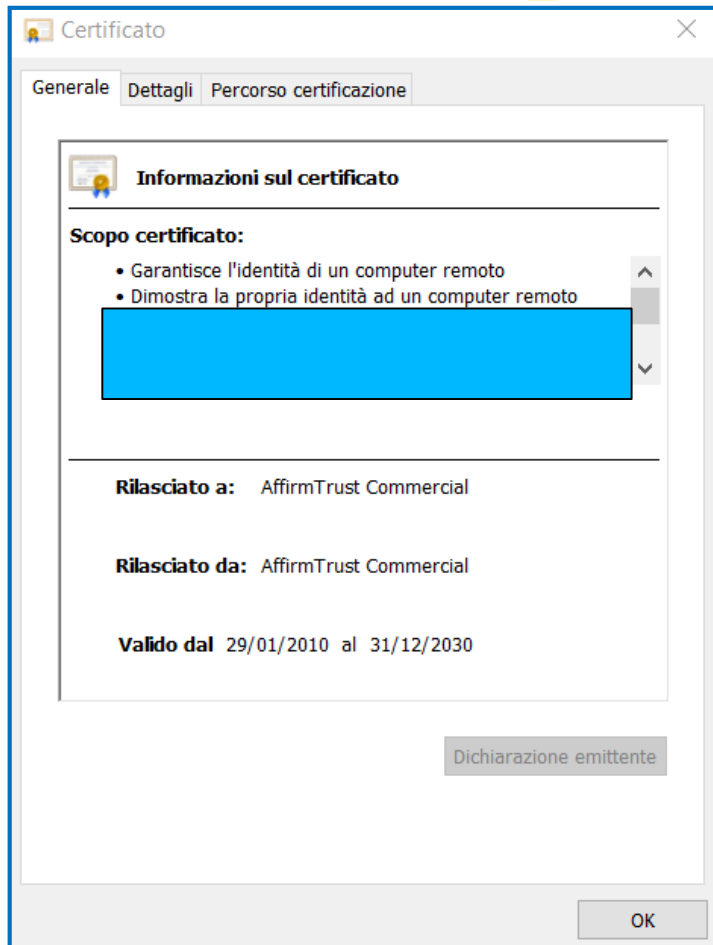
Windows (I)



Windows (II)



Windows (III)



PersonalSet: Caso 1



1. $KPRIV-S + CERT(S=Subject-S, KPUB=KPUB-S, I=Subject-S)$
2. Contatto Certification Authority $CA-X$
 - ☐ Tipicamente presento file di formato standard (Certificate Signing Request: CSR)
 - ☐ Dimostro di "avere diritto" a $Subject-S$
3. $KPRIV-S + CERT(S=Subject-S, KPUB=KPUB-S, I=CA-X)$

PersonalSet: Caso 2

1. Contatto Certification Authority CA-X

- Dimostro di "avere diritto" a Subject-S

2. CA-X mi consegna

KPRIV-S + CERT(S=Subject-S, KPUB=KPUB-S, I=CA-X)

- Caso molto più comune in pratica (più semplice per gli utenti)
 - Consegna avviene su smartcard o chiave USB
- Colab Notebook mostrano caso 1

Google Colab Notebook



□ Digital Signatures

Inizializzazione KeySet / TrustSet

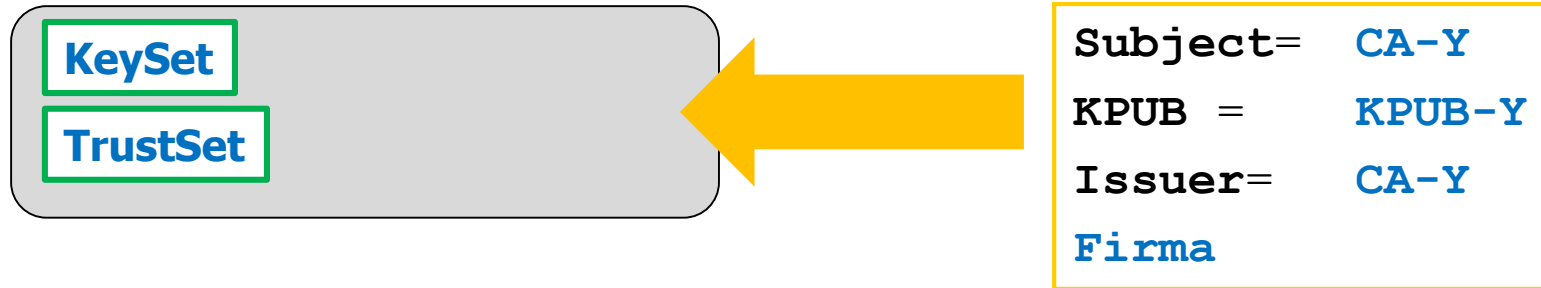


KeySet, TrustSet: Inizializzazione?

- Inizializzazione:
 - Vediamo dopo

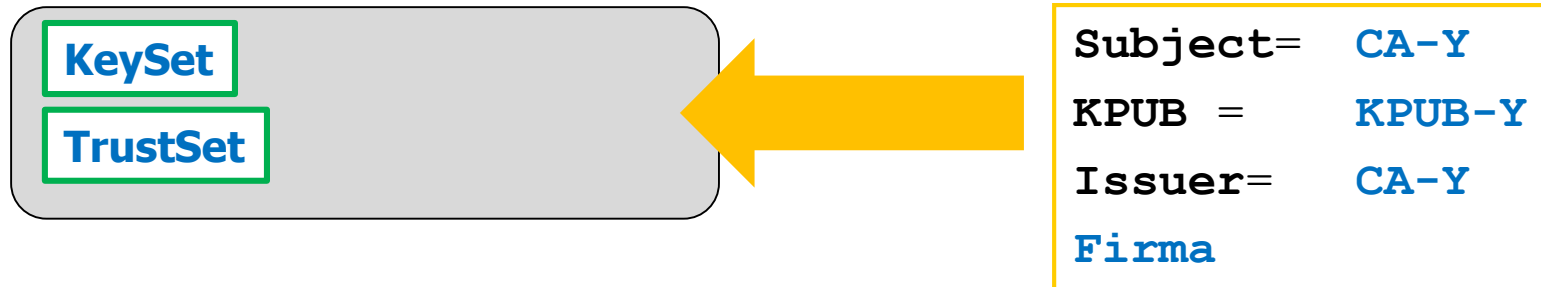


Tentativo (I)



- ❑ KeySet / TrustSet vuoto
 - ❑ Oppure con CA diverse da CA-Y
- ❑ Ricevo certificato self-signed da canale con network attacker

Tentativo (II)



- ❑ KeySet / TrustSet vuoto o con CA diverse da CA-Y
- ❑ Ricevo certificato self-signed da canale con network attacker
- ❑ Verifico `CERT.Firma` usando `CERT.KPUB`
- ❑ Se ha successo allora CERT è autentico e integro, quindi
 - ❑ Posso mettere CERT in KeySet
 - ❑ Se mi fido di CA-Y allora posso mettere CERT anche in TrustSet

Dato di fatto

- ❑ **Chiunque** può costruire un certificato self-signed con Subject **arbitrario**
- ❑ E' solo una sequenza di byte con formato standard

```
Subject=    "Bartoli Alberto"  
KPUB  =     KPUB-1  
Issuer=     "Bartoli Alberto"
```

```
Subject=    "www.unicredit.it"  
KPUB  =     KPUB-3  
Issuer=     "www.unicredit.it"
```

```
Subject=    "esse3.units.it"  
KPUB  =     KPUB-2  
Issuer=     "esse3.units.it"
```

Esempio

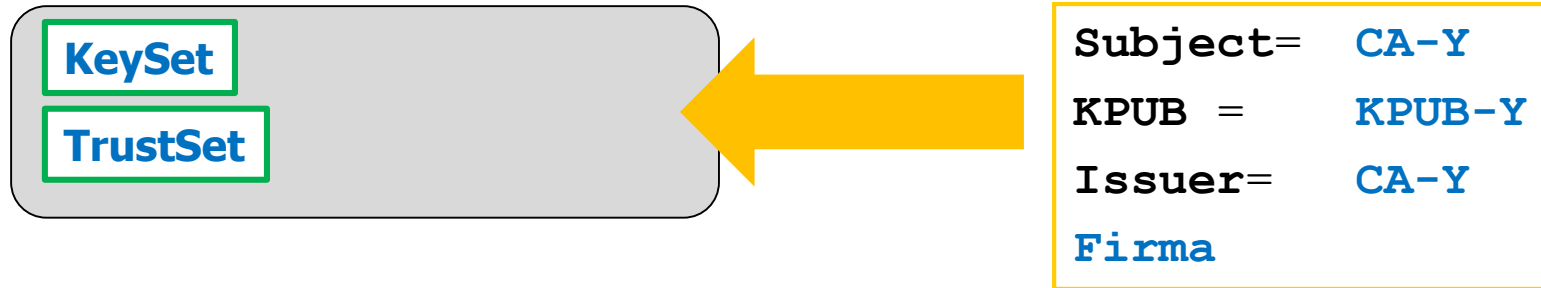
```
C:\>keytool -genkey -alias fakekey
Enter keystore password: 
What is your first and last name?
[Unknown]:
What is the name of your organizational unit?
[Unknown]: Class 4 Public Primary Certification Authority
What is the name of your organization?
[Unknown]: Verisign Inc.
What is the name of your City or Locality?
[Unknown]:
What is the name of your State or Province?
[Unknown]:
What is the two-letter country code for this unit?
[Unknown]: US
Is <CN=Unknown, OU=Class 4 Public Primary Certification Authority, O=Verisign Inc., L=Unknown, ST=Unknown, C=US> correct?
[no]: yes

Enter key password for <fakekey>
(RETURN if same as keystore password):

C:\>
```

Certificate name:	Issuer:
VeriSign, Inc. Class 4 Public Primary Certification Authority US	VeriSign, Inc. Class 4 Public Primary Certification Authority US
Certificate Version : 1 Serial Number : -1 Not Valid Before : Jan 29 00:00:00 1996 GMT Not Valid After : Dec 31 23:59:59 1999 GMT Fingerprint: B5 46 6D EB 70 02 18 9A 2E BF 13 BC 59 51 23 AB Public key algorithm : rsaEncryption	

Conseguenza



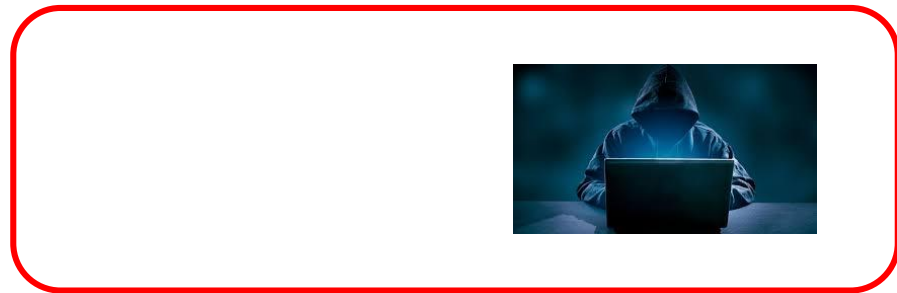
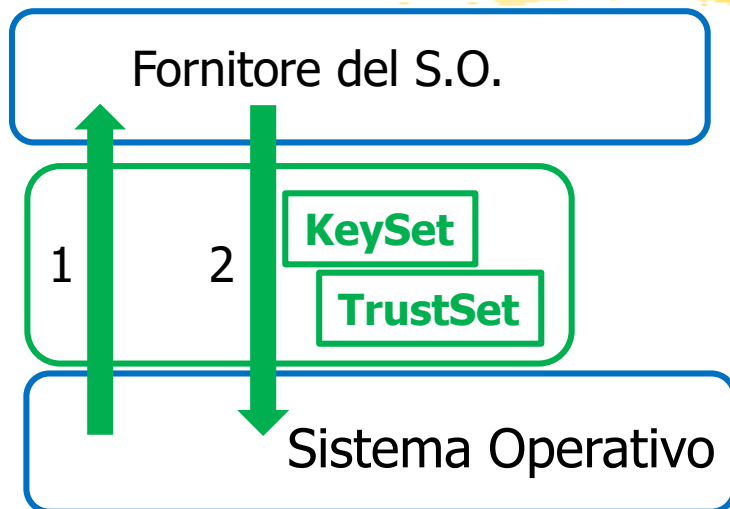
- ❑ KeySet / TrustSet vuoto o con CA diverse da CA-Y
- ❑ Ricevo certificato self-signed da canale con network attacker
- ❑ Verifico CERT.Firma usando CERT.KPUB
- ❑ Se ha successo allora CERT è autentico e integro, quindi non ho nessuna garanzia su chi lo ha generato
 - ~~❑ Posso mettere CERT in KeySet~~
 - ~~❑ Se mi fido di CA-Y allora posso mettere CERT anche in TrustSet~~

Google Colab Notebook



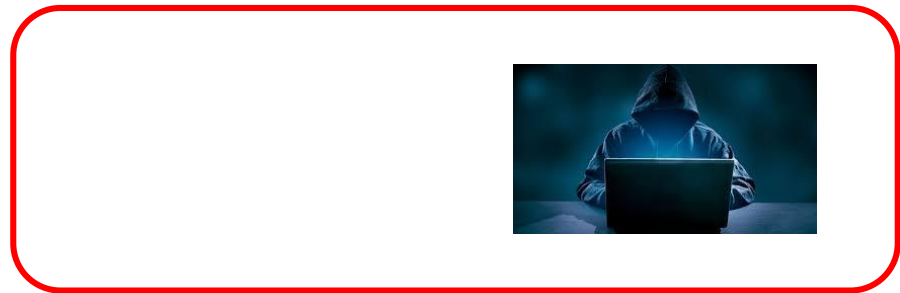
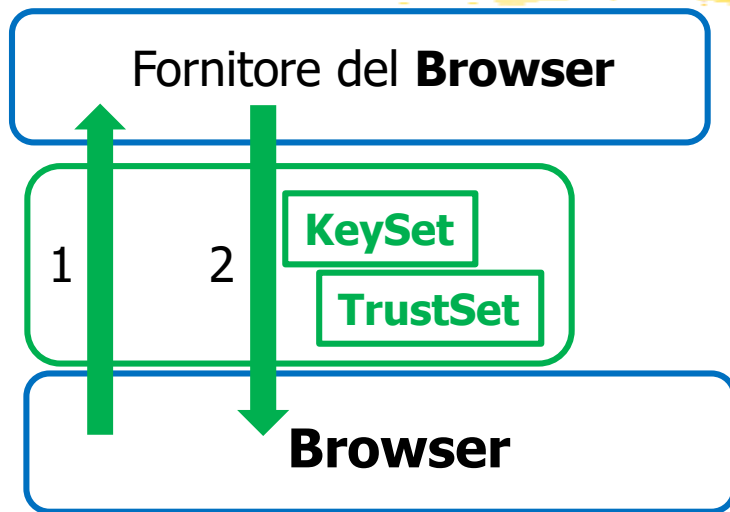
□ Self-signed Certificates

Inizializzazione KeySet / TrustSet (I-a)



- Sono parte del Sistema Operativo
 - Tutte le applicazioni usano KeySet/TrustSet del s.o.
- **Mi fido** del fornitore del Sistema Operativo
- **Mi fido** di avere ricevuto S.O. su **canale sicuro**
(che devo usare solo una volta)

Inizializzazione KeySet / TrustSet (I-b)



- ❑ Eccezione frequente: **Browser** spesso usano **il proprio** KeySet/TrustSet
- ❑ **Mi fido** del fornitore del Browser
- ❑ **Mi fido** di avere ricevuto Browser su **canale sicuro** (che devo usare solo una volta)

Importantissimo 1



- ❑ **Mi fido** del fornitore del software
 - ❑ Assumo che non mi fornisca KeySet/TrustSet fraudolenti
- ❑ E' una "**fiducia molto forte**"
 - ❑ Ha un potere enorme
- ❑ Una "fortissima fiducia" nel fornitore del sw
è indispensabile in ogni caso
 - ❑ BIG problem: supply chain

Importantissimo 2



- ❑ Indispensabile che il **canale** su cui arrivano KeySet / TrustSet sia **veramente sicuro**
- ❑ Il canale su cui ricevo il sw **deve** essere sicuro **a prescindere** da KeySet /TrustSet

Ricezione self-signed (I)

- ❑ Talvolta i sw **ricevono** da canale non fidato **certificati self-signed**
 - ❑ Quando si descrivono questi scenari si assume implicitamente $\text{Issuer} \notin \text{KeySet/TrustSet}$
 - ❑ Altrimenti basta vedere se è identico a quello che abbiamo già
- ❑ Utente deve decidere cosa fare

- ❑ Caso 1
 - ❑ Uso il certificato per completare l'operazione e poi lo dimentico
- ❑ Caso 2
 - ❑ Inserisco il certificato in KeySet/TrustSet

Ricezione self-signed (II)

- ❑ Caso 1 Uso il certificato per completare l'operazione e poi lo dimentico
 - ❑ Approccio errato: *"Mai usare un cert. self-signed"*
 - ❑ Approccio corretto: *"Prima di usare un cert. self-signed pensarci bene."*
 - ❑ *Mi fido del modo in cui mi è arrivato?*
 - ❑ *Cosa mi costa non accettarlo?*
 - ❑ *Cosa mi costa accettarlo e poi scoprire che è fraudolento?"*
- ❑ Caso 2 Inserisco il certificato in KeySet/TrustSet
 - ❑ Approccio corretto: *"Mai usare un cert. self-signed"*
(a meno che non stia **installando un sw** fidato,
ricevuto su canale fidato)

Aveva ragione o no?

NON risolvono “automaticamente” i problemi di security !

I do not know to whom should be credited the important truth
"Whenever anyone says that a problem is easily solved by
cryptography, it shows that he doesn't understand it".

Roger Needham

quasi equivalente a
(quasi)



Test – Firma e Certificati

